Travaux Dirigés 1 : Introduction à NumPy

ali.zainoul.az@gmail.com

23 mars 2026

Introduction

NumPy (Numerical Python) est une bibliothèque fondamentale pour le calcul scientifique en Python. Elle permet de manipuler efficacement des tableaux multidimensionnels et propose des fonctions mathématiques optimisées. Ce TD vous guidera à travers les fonctionnalités essentielles de NumPy, avec des exemples concrets et des exercices progressifs.

Aide-mémoire NumPy

Fonctions de création de tableaux

Fonction	Usage
`np.array([...])`	Crée un tableau à partir d'une liste Python
`np.arange(start, stop, step)`	Crée un tableau de valeurs régulièrement espacées
`np.linspace(start, stop, num)`	Crée un tableau de valeurs linéairement espacées
`np.zeros(shape)`	Crée un tableau de zéros de forme donnée
`np.ones(shape)`	Crée un tableau de uns de forme donnée
`np.eye(n)`	Crée une matrice identité de taille $n \times n$
`np.full(shape, value)`	Crée un tableau rempli avec une valeur constante
`np.random.rand(d0, d1, ...)`	Crée un tableau de nombres aléatoires uniformes dans [0, 1)
`np.random.randint(low, high, size)`	Crée un tableau d'entiers aléatoires

Opérations sur les tableaux

Fonction	Usage
`np.sum(arr, axis)`	Somme des éléments le long d'un axe
`np.mean(arr, axis)`	Moyenne des éléments le long d'un axe
`np.std(arr, axis)`	Écart-type des éléments le long d'un axe
`np.max(arr, axis)`	Maximum des éléments le long d'un axe
`np.min(arr, axis)`	Minimum des éléments le long d'un axe
`np.argmax(arr, axis)`	Indice du maximum le long d'un axe
`np.argmin(arr, axis)`	Indice du minimum le long d'un axe
`np.reshape(arr, newshape)`	Change la forme du tableau
`arr.T`	Transpose une matrice

Fonctions mathématiques

Fonction	Usage
`np.sin(x)`	Calcule le sinus de chaque élément de x
`np.cos(x)`	Calcule le cosinus de chaque élément de x
`np.exp(x)`	Calcule l'exponentielle de chaque élément de x
`np.log(x)`	Calcule le logarithme naturel de chaque élément de x
`np.sqrt(x)`	Calcule la racine carrée de chaque élément de x
`np.round(x, decimals)`	Arrondit chaque élément de x à `decimals` décimales

1 Exercice 1 : Premiers pas avec NumPy

Objectif : Découvrir l'importation de NumPy, la création de tableaux, et l'exploration de leurs propriétés.

1.1 Installation et première prise en main

- **Importer NumPy** : Utilisez la convention `import numpy as np`.
- **Vérifier la version installée** : Affichez `np.__version__`.
- **Créer un tableau contenant les entiers de 0 à 9** : Utilisez `np.arange`.
- **Identifier ses caractéristiques principales** : Utilisez `ndim`, `shape`, `size`, et `dtype`.

Remarque : `np.arange` est similaire à `range` de Python, mais retourne un tableau NumPy.

1.2 Création de tableaux

- **Créer une matrice de zéros de taille 3×4** : Utilisez `np.zeros`.
- **Créer une matrice de uns de taille 4×3** : Utilisez `np.ones`.
- **Créer une matrice identité de taille 5×5** : Utilisez `np.eye`.
- **Créer une matrice remplie avec une valeur constante** : Utilisez `np.full`.

Conseil : Les matrices de zéros et de uns sont souvent utilisées pour initialiser des tableaux avant de les remplir.

2 Exercice 2 : Indexation et extraction

Objectif : Maîtriser l'accès aux éléments d'un tableau et l'extraction de sous-tableaux.

2.1 Manipulation de tableaux 1D

- **Créer un tableau contenant les entiers de 0 à 19** : Utilisez `np.arange`.
- **Extraire une sous-partie du tableau (indices 5 à 15)** : Utilisez le *slicing* `[start:end]`.
- **Extraire un élément sur deux** : Utilisez un pas dans le *slicing* : `[::2]`.
- **Inverser l'ordre du tableau** : Utilisez `[::-1]`.

Exemple : `arr[5:16]` extrait les éléments des indices 5 à 15 (inclus).

2.2 Manipulation de matrices

- **Créer une matrice 4×4 contenant les valeurs de 1 à 16** : Utilisez `np.arange` suivi de `reshape`.
- **Extraire une ligne complète** : Utilisez `matrix[i, :]`.
- **Extraire une colonne complète** : Utilisez `matrix[:, j]`.
- **Extraire une sous-matrice centrale** : Utilisez le *slicing* sur les deux dimensions : `matrix[1:3, 1:3]`.

Remarque : En NumPy, les indices commencent à 0.

3 Exercice 3 : Calcul vectorisé

Objectif : Découvrir les opérations mathématiques sur les tableaux sans utiliser de boucles.

3.1 Opérations élémentaires

- **Créer deux tableaux de même taille** : Utilisez `np.array`.
- **Effectuer les opérations : addition, multiplication** : Les opérations s'appliquent élément par élément.

— **Calculer le carré des éléments** : Utilisez l'opérateur `**`.
— **Calculer la racine carrée** : Utilisez `np.sqrt`.

Exemple : `a + b` additionne les éléments de `a` et `b` un à un.

3.2 Introduction au broadcasting

— **Créer une matrice 2D** : Utilisez `np.arange` et `reshape`.
— **Ajouter une constante à tous les éléments** : NumPy étend la constante à la taille de la matrice.
— **Multiplier chaque ligne par un coefficient différent** : Utilisez un tableau de coefficients de même longueur que le nombre de lignes.
— **Normaliser les colonnes (centrer-réduire)** : Soustrayez la moyenne et divisez par l'écart-type de chaque colonne.

Conseil : Le *broadcasting* permet d'éviter des boucles explicites et rend le code plus concis et efficace.

4 Exercice 4 : Statistiques

Objectif : Utiliser NumPy pour calculer des statistiques descriptives sur des tableaux.

4.1 Statistiques descriptives

— **Générer un tableau de valeurs aléatoires** : Utilisez `np.random.randn`.
— **Calculer moyenne, médiane, variance et écart-type** : Utilisez `np.mean`, `np.median`, `np.var`, et `np.std`.
— **Identifier les valeurs extrêmes** : Utilisez `np.min` et `np.max`.
— **Calculer les quartiles** : Utilisez `np.percentile`.

Exemple : `np.mean(data)` calcule la moyenne des éléments de `data`.

4.2 Statistiques sur matrices

— **Créer une matrice de valeurs aléatoires** : Utilisez `np.random.randint`.
— **Calculer la moyenne de chaque ligne** : Utilisez `np.mean` avec `axis=1`.
— **Calculer la somme de chaque colonne** : Utilisez `np.sum` avec `axis=0`.
— **Identifier la position des maximums** : Utilisez `np.argmax`.

Remarque : L'argument `axis` permet de spécifier la dimension le long de laquelle l'opération est appliquée.

5 Exercice 5 : Filtrage avec masques

Objectif : Utiliser des masques booléens pour filtrer et manipuler les données.

5.1 Filtrage simple

— **Générer un tableau d'entiers aléatoires** : Utilisez `np.random.randint`.
— **Créer un masque pour filtrer certaines valeurs** : Un masque est un tableau booléen de même forme que le tableau à filtrer.
— **Extraire les valeurs correspondantes** : Utilisez le masque comme indice : `values[mask]`.
— **Compter le nombre d'éléments filtrés** : Utilisez `np.sum` sur le masque.

Exemple : `mask = values > 50` crée un masque où chaque élément est `True` si la valeur est supérieure à 50.

5.2 Conditions multiples

- **Générer des valeurs aléatoires** : Utilisez `np.random.randn`.
- **Filtrer les valeurs dans un intervalle donné** : Combinez des masques avec les opérateurs logiques `&`, `|`, et .
- **Identifier les valeurs hors intervalle** : Utilisez l'opérateur pour inverser un masque.
- **Remplacer ces valeurs par une statistique (ex : moyenne)** : Utilisez le masque pour identifier les valeurs à remplacer.

Conseil : Les opérateurs logiques doivent être protégés par des parenthèses : `(mask1) & (mask2)`.

6 Exercice 6 : Transformation de tableaux

Objectif : Découvrir les opérations de transformation de la structure des tableaux.

6.1 Reshape et transposition

- **Créer un tableau 1D** : Utilisez `np.arange`.
- **Le transformer en matrice** : Utilisez `reshape`.
- **Transposer la matrice** : Utilisez l'attribut `T`.
- **Revenir à un tableau 1D** : Utilisez `flatten`.

Remarque : La transposition échange les lignes et les colonnes d'une matrice.

6.2 Concaténation et découpage

- **Créer plusieurs tableaux** : Utilisez `np.array`.
- **Les concaténer** : Utilisez `np.concatenate`.
- **Les empiler pour former une matrice** : Utilisez `np.vstack` ou `np.hstack`.
- **Les séparer à nouveau** : Utilisez `np.split`.

Exemple : `np.vstack([a, b])` empile verticalement les tableaux `a` et `b`.

7 Exercice 7 : Fonctions mathématiques

Objectif : Découvrir les fonctions mathématiques vectorisées de NumPy.

7.1 Fonctions usuelles

- **Générer un ensemble de valeurs** : Utilisez `np.linspace`.
- **Appliquer des fonctions trigonométriques** : Utilisez `np.sin`, `np.cos`, etc.
- **Calculer exponentielle et logarithme** : Utilisez `np.exp` et `np.log`.
- **Arrondir les résultats** : Utilisez `np.round`.

Conseil : `np.linspace` est utile pour générer des valeurs régulièrement espacées.

7.2 Agrégations avancées

- **Créer une matrice** : Utilisez `np.random.randint`.
- **Calculer des agrégations cumulées** : Utilisez `np.cumsum` et `np.cumprod`.
- **Appliquer une fonction personnalisée sur chaque ligne** : Utilisez `np.apply_along_axis`.

Exemple : `np.cumsum(matrix, axis=1)` calcule la somme cumulative le long des lignes.

8 Exercice 8 : Mini-projet — Analyse de données météo

Objectif : Mettre en pratique les concepts appris sur un cas concret d'analyse de données.

On considère un jeu de données représentant les températures (en °C) mesurées pendant 30 jours dans 5 villes différentes.

- **Générer une matrice de taille (30, 5) avec des températures aléatoires réalistes (entre -5 et 35°C)** : Utilisez `np.random.uniform`.
- **Calculer la température moyenne pour chaque ville** : Utilisez `np.mean` avec `axis=0`.
- **Identifier le jour le plus chaud pour chaque ville** : Utilisez `np.argmax` avec `axis=0`.
- **Calculer la température moyenne globale** : Utilisez `np.mean` sur toute la matrice.
- **Détecter les jours de forte chaleur (température $>$ 30°C)** : Créez un masque booléen.
- **Compter le nombre de jours de canicule par ville** : Utilisez `np.sum` sur le masque.

Conseil : Pour générer des températures réalistes, utilisez une distribution uniforme entre -5 et 35.

Résumé des concepts clés

Création de tableaux

- `np.array`, `np.arange`, `np.linspace` : Création de tableaux à partir de listes ou d'intervalles.
- `np.zeros`, `np.ones`, `np.eye` : Création de tableaux de zéros, de uns, ou de matrices identité.
- `np.random` : Génération de tableaux de valeurs aléatoires.

Indexation et slicing

- `arr[start:end:step]` : Extraction de sous-tableaux.
- `arr[mask]` : Filtrage avec des masques booléens.
- `arr[i, :]`, `arr[:, j]` : Extraction de lignes et colonnes.

Calcul vectorisé et broadcasting

- Les opérations s'appliquent élément par élément.
- Le broadcasting permet d'étendre automatiquement les dimensions pour les opérations.
- Utilisez `np.newaxis` pour ajouter des dimensions.

Statistiques et agrégations

- `np.mean`, `np.std`, `np.sum`, `np.max`, `np.min` : Calcul de statistiques.
- `axis` : Spécifie la dimension le long de laquelle l'opération est appliquée.

Fonctions mathématiques

- `np.sin`, `np.cos`, `np.exp`, `np.log` : Fonctions mathématiques vectorisées.
- `np.round` : Arrondi des valeurs.

Bonnes pratiques

- Toujours vérifier les dimensions des tableaux avec `shape`.
- Utiliser `np.allclose` pour comparer des flottants.
- Éviter les boucles explicites : privilégier le calcul vectorisé.